

Inheritance and Polymorphism

Q167 Case I

=====

```
import javax.swing.*;

class Calculator extends JFrame{

}

class Demo{
    public static void main(String args[]){
        Calculator c1=new Calculator();
        c1.setSize(300,300);
        c1.setTitle("Calculator 1");
        c1.setDefaultCloseOperation(Calculator.EXIT_ON_CLOSE);
        c1.setVisible(true);

        JFrame c2=new JFrame();
        c2.setSize(300,300);
        c2.setTitle("Calculator2");
        c2.setDefaultCloseOperation(Calculator.EXIT_ON_CLOSE);
        c2.setVisible(true);
    }
}

//Calculator IS-A JFrame
//Calculator -->Subclass
//JFrame-->Superclass
```

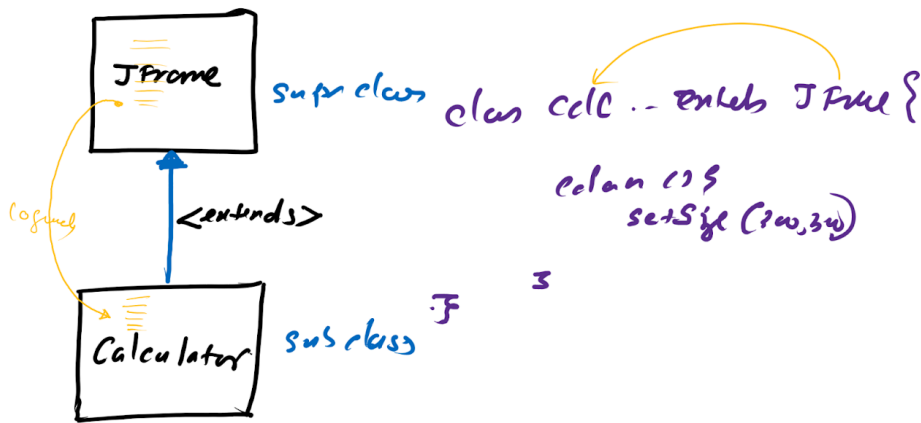
Q168 Case III

=====

```
import javax.swing.*;

class Calculator extends JFrame{
    Calculator(){
        setSize(300,300);
        setTitle("Calculator");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
}

class Demo{
    public static void main(String args[]){
        Calculator c1=new Calculator();
        c1.setVisible(true);
    }
}
```



Q169 From Q168

```

class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
}

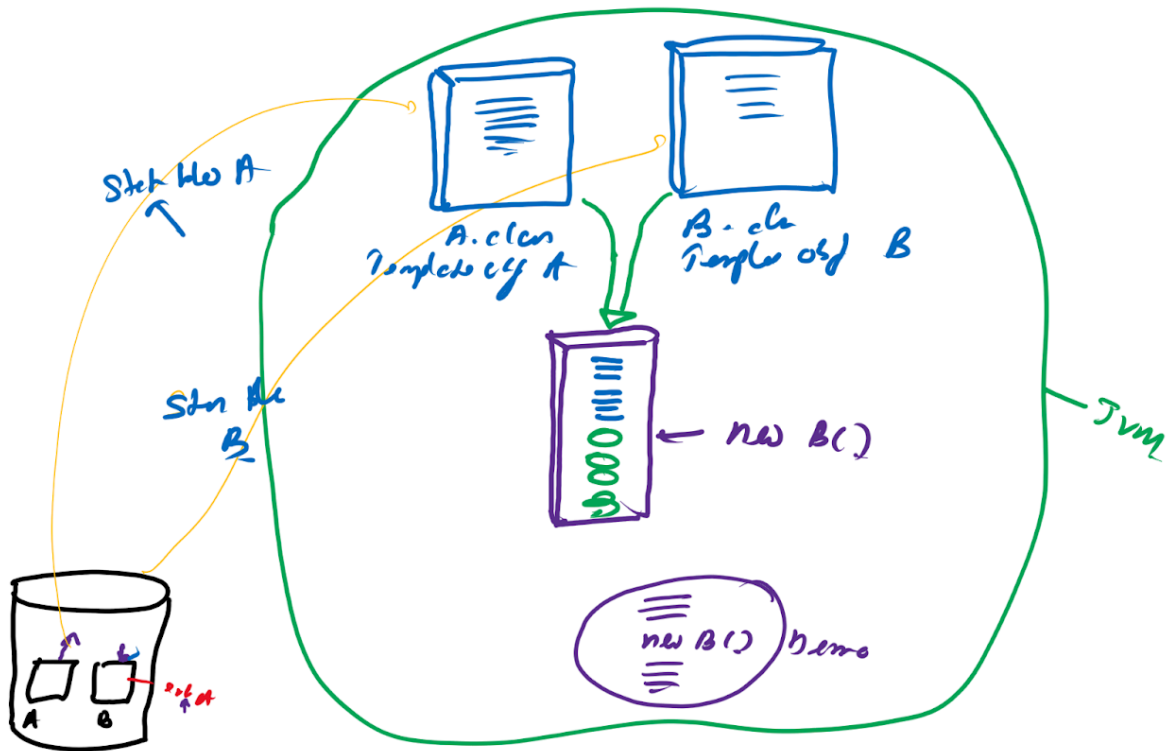
class B extends A{
    int b;
    public void mB(){
        System.out.println("mB : "+b);
    }
    public void callmAB(){
        mA(); //Legal ?
        mB();
    }
    public void printAB(){
        System.out.println("a : "+a); //Legal ?
        System.out.println("b : "+b);
    }
}
  
```

////////////////////////////////////

Q170 Case III

=====

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    static{
        System.out.println("Static block A");
    }
}
class B extends A{
    int b;
    public void mB(){
        System.out.println("mB : "+b);
    }
    static{
        System.out.println("Static block B");
    }
}
class Demo{
    public static void main(String args[]){
        new B();
    }
}
```



```
////////////////////////////////////  
Q171 Exercise  
=====  
class A{  
    int a;  
    static{  
        System.out.println("Static block A");  
    }  
}  
class B extends A{  
    int b;  
    static{  
        System.out.println("Static block B");  
    }  
}  
class C extends B{  
    int c;  
    static{  
        System.out.println("Static block C");  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        new C();  
    }  
}
```

```
////////////////////////////////////  
Q172 Exercise  
=====  
class A{  
    int a;  
    static{  
        System.out.println("Static block A");  
    }  
}  
class B extends A{  
    int b;  
    static{  
        System.out.println("Static block B");  
    }  
}  
class C extends B{  
    int c;  
    static{  
        System.out.println("Static block C");  
    }  
}
```

```
class D extends B{
    int d;
    static{
        System.out.println("Static block D");
    }
}

class Demo{
    public static void main(String args[]){
        new D();
    }
}
```

////////////////////////////////////

Q173 Case IV

=====

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
    A(int i, int j){
        System.out.println("A(int,int)");
    }
}

class B extends A{
    int b;
    B(){
        System.out.println("B()");
    }
    B(int i){
        System.out.println("B(int)");
    }
    B(int i, int j){
        System.out.println("B(int,int)");
    }
    public void mB(){
        System.out.println("mB : "+b);
    }
}
```

```
class B extends A{
    int b;
    B(){
        System.out.println("B()");
    }
    B(int i){
        System.out.println("B(int)");
    }
    B(int i, int j){
        System.out.println("B(int,int)");
    }
    public void mB(){
        System.out.println("mB : "+b);
    }
}
```

```
class Demo{
    public static void main(String args[]){
        new B();
        System.out.println("-----");
        new B(100);
        System.out.println("-----");
        new B(100,200);
    }
}
```

////////////////////////////////////

Q174 From Q173

=====

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
    A(int i, int j){
        System.out.println("A(int,int)");
    }
}
```

```
class B extends A{
    int b;
    B(){
        //super(); //Compiler inserts
        System.out.println("B()");
    }
    B(int i){
        //super(); //Compiler inserts
        System.out.println("B(int)");
    }
    B(int i, int j){
        //super(); //Compiler inserts
        System.out.println("B(int,int)");
    }
    public void mB(){
        System.out.println("mB : "+b);
    }
}
```

}

```
class Demo{
    public static void main(String args[]){
        new B();
        System.out.println("-----");
        new B(100);
        System.out.println("-----");
        new B(100,200);
    }
}
```

////////////////////////////////////

Q175 From Q174

=====

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
    A(int i, int j){
        System.out.println("A(int,int)");
    }
}
```

```
class B extends A{
    int b;
    B(){
        super();
        System.out.println("B()");
    }
    B(int i){
        super(i);
        System.out.println("B(int)");
    }
    B(int i, int j){
        super(i,j);
        System.out.println("B(int,int)");
    }
    public void mB(){
        System.out.println("mB : "+b);
    }
}
```



```
class Demo{
    public static void main(String args[]){
        A a1=null;
        a1.a=100;
        a1.b=200; //Illegal
        a1.mA();
        a1.mB(); //Illegal
        //-----
        B b1=null;
        b1.a=100; //Legal
        b1.b=200;
        b1.mA(); //Legal
        b1.mB();

    }
}
```

```
class Demo{
    public static void main(String args[]){
        A a1=new B(); //Legal
        a1.a=100;
        a1.b=200; //Illegal

        B b1=new A(); //Illegal
    }
}
```

Q180 Case VII (Method Overriding)

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    public void print(){
        System.out.println("print() of A");
    }
}

class B extends A{
    int b;
    public void mB(){
        System.out.println("mB : "+b);
    }
    public void print(){ //Overriding [Same name, same signature (Parameter list)]
        System.out.println("print() of B");
    }
}

class Demo{
    public static void main(String args[]){
        B b1=new B();
        b1.print(); //print() of B
    }
}
```

```
class A{
    int a;
}
class B extends A{
    int b;
}
class C extends A,B{} //Illegal
```

////////////////////////////////////

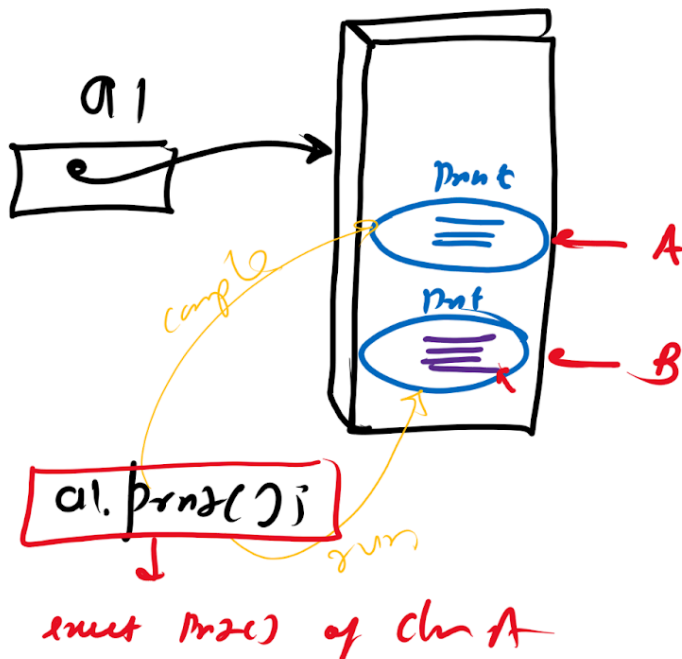
Q183 Case X (Dynamic Method dispatch)

=====

```
class A{
    int a;
    public void mA(){
        System.out.println("mA : "+a);
    }
    public void print(){
        System.out.println("print() of A");
    }
}

class B extends A{
    int b;
    public void mB(){
        System.out.println("mB : "+b);
    }
    public void print(){
        System.out.println("print() of B");
    }
}

class Demo{
    public static void main(String args[]){
        A a1=new B();
        a1.print();
    }
}
```



Q184

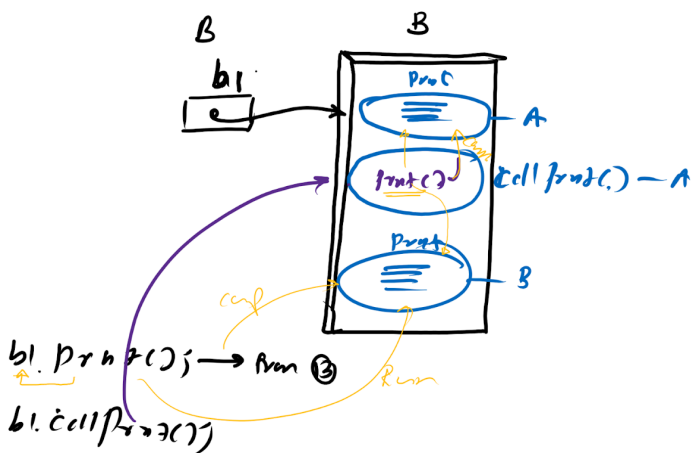
}

Q185

}

=====

}



Polymorphism (Single interface / many forms)

Compile time poly
Compiler
Method overloading
(Compile time)
(Static binding)

Runtime poly -
JVM
Method overriding
(At runtime)
dynamic binding (Method dispatch)

Q187 Compile time polymorphism

```
=====
class Demo{
    public static void main(String args[]){
        int a=-100;
        long b=-23;
        float c=-34.55f;
        double d=-56.344;

        a=Math.abs(a); //single interface->"abs"
        b=Math.abs(b);
        c=Math.abs(c);
        d=Math.abs(d);

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
        System.out.println(d);
    }
}
```

Q188 Runtime Polymorphism

```
class Car{
    public void park(){
        System.out.println("Car Parking");
    }
}

class Bus{
    public void park(){
        System.out.println("Bus Parking");
    }
}

class Van{
    public void park(){
        System.out.println("Van Parking");
    }
}

class Demo{
    public static void main(String args[]){
        v1=new Car();
        v1.park(); //Car Parking

        v1=new Bus();
        v1.park(); //Bus Parking

        v1=new Van();
        v1.park(); //Van Parking
    }
}
```

////////////////////////////////////

```
class Vehicle{
    public void park(){
        System.out.println("Vehicle Parking");
    }
}

class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking");
    }
}
```

```

class Bus extends Vehicle{
    public void park(){
        System.out.println("Bus Parking");
    }
}
class Van extends Vehicle{
    public void park(){
        System.out.println("Van Parking");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1;

        v1=new Car();
        v1.park(); //Car Parking

        v1=new Bus();
        v1.park(); //Bus Parking

        v1=new Van();
        v1.park(); //Van Parking
    }
}

```

//////////////////////////////////////
 Q190

```

class Vehicle{
    public void park(){
        System.out.println("Vehicle Parking");
    }
}
class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking");
    }
}
class Bus extends Vehicle{
    public void park(){
        System.out.println("Bus Parking");
    }
}
class Van extends Vehicle{
    public void park(){
        System.out.println("Van Parking");
    }
}

```

////////////////////////////////////

=====

```
import java.util.*;
```

```
class Alarm{
```

```
public void operateAlarm(int waterLevel){
    System.out.println(waterLevel>=50 ? "ON":"OFF");
}
```

}

}

}

```
class ControlRoom{
    private Alarm alarm;
    private Display display;
    private SMSSender sMSSender;

    private int waterLevel;

    public void addAlarm(Alarm alarm){
        this.alarm=alarm;
    }
    public void addDisplay(Display display){
        this.display=display;
    }
    public void addSMSSender(SMSSender sMSSender){
        this.sMSSender=sMSSender;
    }
}
```



```

class SMSSender{
    public void sendSMS(int waterLevel){
        System.out.println("Sending SMS : "+waterLevel);
    }
}

class ControlRoom{
    private Alarm alarm;
    private Display display;
    private SMSSender sMSSender;
    private Splitter splitter;

    private int waterLevel;

    public void addSplitter(Splitter splitter){
        this.splitter=splitter;
    }

    public void addAlarm(Alarm alarm){
        this.alarm=alarm;
    }
    public void addDisplay(Display display){
        this.display=display;
    }
    public void addSMSSender(SMSSender sMSSender){
        this.sMSSender=sMSSender;
    }
    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
            notifyObjects();
        }
    }
    public void notifyObjects(){
        alarm.operateAlarm(waterLevel);
        display.display(waterLevel);
        sMSSender.sendSMS(waterLevel);
        splitter.split(waterLevel);
    }
}

class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom= new ControlRoom();
        controlRoom.addAlarm(new Alarm());
        controlRoom.addDisplay(new Display());
        controlRoom.addSMSSender(new SMSSender());
        controlRoom.addSplitter(new Splitter());

        Random r=new Random();
    }
}

```

////////////////////////////////////

=====

```
import java.util.*;
```



```

class SMSSender extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending SMS : "+waterLevel);
    }
}

class ControlRoom{

    private WaterLevelObserver[] observerArray=new WaterLevelObserver[100];
    private int nextIndex;

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        observerArray[nextIndex++]=ob;
    }
    //removeWaterLevelObserver

    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
            notifyObjects();
        }
    }

    public void notifyObjects(){
        for(int i=0; i<nextIndex; i++){
            observerArray[i].update(waterLevel);
        }
    }
}

class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom= new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new Display());
        controlRoom.addWaterLevelObserver(new Splitter());

        Random r=new Random();
        while(true){
            int waterLevel=r.nextInt(101);//0 to 100
            controlRoom.setWaterLevel(waterLevel);
            try{Thread.sleep(1000);}catch(Exception ex){}
        }
    }
}

```